CHANGELOG

Unreleased

- Fixed the taskbar identity produced by the start-launcher build: a name that already begins with the brand no longer gets a second `Audion.Tools.` prefix, so Hub Manager is `Audion.Hub.Manager` instead of `Audion.Tools.Audion.Hub.Manager`. Windows groups taskbar windows by this id.
- Built `Start.exe` from the existing start-launcher sources and gave the app a new icon: a white three-armed hub mark on a near-black rounded tile, rendered at 16/24/32/48/64/128/256 with a simplified variant below 32px, where three separate nodes collapse into a blur. The previous icon is kept in `backup/icons_previous/`.
- Routed `git bundle create` and `git pull --ff-only` through the Git engine instead of composing shell strings in the GUI. The bundle command interpolated a filesystem path into the command line, so a folder name containing `&` or a quote would have broken or mangled it; both now run as argument lists. The engine functions existed all along and were simply never called.
- Removed two Forgejo API helpers that nothing used (`get_repo`, `list_public_keys`).
- Stopped a hand-edited config from preventing startup: a single stray comma in `config/projects.json` used to abort the application with a traceback before the window opened, because the registry was read with a raising JSON loader on import. Startup config reads now fall back to defaults, keep the remaining valid records, and report the problem in the terminal dock. An empty or unreadable registry yields a usable "unconfigured" project pointing at the Hub Manager folder, so the GUI opens and the config can be fixed from inside it. Version bumped to `1.10.0`.
- Re-validated projection plans at apply time: `apply_projection_plan` trusted the roots stored in the plan JSON, so a stale or edited plan could delete under a path that overlaps the source. The profile and the non-overlapping-roots rule are now re-checked before anything is written or removed.
- **Behaviour change:** copy or hash errors now block the delete phase unconditionally. Previously `delete_after_successful_copy: false` in a profile let deletion proceed after failures, which is

exactly the case where a mirror can drop files that exist nowhere else. The setting remains in the config but can no longer disable that protection.

- Fixed the token workflow, which required saving before anything worked at all: every button read the token back from the credential store, so a token typed into the field did nothing until it was stored. The typed token is now used immediately and saving became what its name suggests — remembering it across restarts. The button is `remember token`, and the status line says which token the buttons are about to use. Version bumped to `1.9.0`.
- Turned forgetting a token into an icon next to the field, and made it clear both places at once: the field and the remembered copy in the credential store.
- Made `git clone` fail fast instead of hanging: it now runs with `GIT_TERMINAL_PROMPT=0`, because a credential prompt has no way to reach the user from the GUI. Cloning over HTTPS with a token that is only in the field now warns that git reads the store, not the form.
- Added a `git clone` button to the `Remote` pane, which had no cloning action at all: it takes the Remote URL field, clones next to the active project folder, refuses to clone onto an existing folder, and runs git as an argument list with `--` rather than through a shell. Version bumped to `1.8.0`.
- Made the `Remote` pane remember the selected platform and URL type between sessions, so it opens where it was left instead of resetting to GitHub.
- Corrected the token scopes a second time, after per-operation testing on a live Forgejo 15: scopes follow the route, not the object. Creating a repository sits under `/api/v1/user/*` and therefore needs `write:user`, not `write:repository`. Git over HTTPS is judged separately and needs a repository scope regardless, so one set of scopes cannot cover both the API and git. Version bumped to `1.7.3`.
- Documented that a credential-bearing clone URL makes Git write the token in the clear into `.git/config` of the working copy, where it survives every backup and mirror of that folder — the same family of mistake as a secret in a URL query parameter, and avoided entirely by the credential-helper path.
- Closed a command-injection path: `urlparse` keeps `;`, `&&` and `$(...)` inside a hostname, and that hostname was interpolated into the `ssh -T <user>@<host>` probe, which the GUI runs through a shell. A `forgejo_hosts.json` arriving with a cloned project could therefore run a command on the first probe. Hostname, SSH user and SSH port are now validated where every path converges, hostile records are dropped from the config instead of loaded, and URLs embedding credentials are refused outright. Version bumped to `1.7.2`.
- Refused remote URLs that make Git execute a command (`ext::`, `fd::`), URLs starting with `-` that Git would read as an option, and URLs carrying control characters. Validation runs both when

saving from the Remote pane and when applying `remotes.json`, so a config file from another project cannot smuggle one in.

- Warned when a Forgejo host is reached over plain HTTP before a token is sent: legitimate for a LAN instance, but the token crosses the wire unencrypted.
- Fixed a sign-in defect that dropped a server typed into the Remote form: the config write went through `remember_login`, which returns early when the login already matches, so the host was never written to `forgejo_hosts.json` and had to be retyped on the next start. Sign-in now persists the host record outright. Version bumped to `1.7.1`.
- Made `forgejo_hosts.json` parsing fault-tolerant: a hand-edited file with a syntax error used to raise straight into a NiceGUI event handler, because the Remote pane reads it on every platform switch and keystroke in the server field.
- Stopped silently truncating the repository list: paging stops at 1000 entries, and reaching that ceiling is now reported instead of presenting a partial list as complete.
- Rejected an unusable SSH port instead of quietly substituting 22, which built a plausible-looking URL that failed later at push time.
- Blocked saving a remote URL with an embedded login or token into `config/remotes.json`, which is a committed file — the secret would have been published with the next commit.
- Required an account login before erasing a stored token: credential helpers key entries by host *and* username, so an empty login matched nothing while reporting success.
- Added Forgejo/Gitea support for self-hosted instances: `config/forgejo_hosts.json` describes servers (address, SSH user/port, preferred URL type, cached login) and holds no secrets; `forgejo_api.py`, `git_credentials.py` and `forgejo_service.py` implement API v1 access, credential-store handling and sign-in. Version bumped to `1.7.0`.
- Adopted the ordinary Forgejo account contract: a personal access token sent as `Authorization: token <TOKEN>`, verified against `/api/v1/user` and then handed to the user's Git credential helper via `git credential approve`. Nothing is written into project config, and plain `git push` over HTTPS picks the same credential up. OAuth2/OIDC sign-in was evaluated and rejected for now: Forgejo has not implemented OAuth2 scopes, so such tokens carry administrative account rights, expire quickly and cannot serve `git push` unaided.
- Corrected the documented token scopes after testing a live Forgejo 15 instance: `/api/v1/user` and `/api/v1/user/repos` require `read:user`, which the repository scopes do not cover. A token holding only `read:repository` answers both with `403 ... required scope(s): [read:user]`. Minimum working set is `read:user` + `read:repository`, plus `write:repository` to create repositories.

- Separated 403 from 401 in every Forgejo call: an insufficient scope now reports "the token is valid but was created without the scope this call needs", names the scope Forgejo asks for, and suggests the working set, instead of showing a bare status code alongside a token that is in fact correct.
- Redacted server error text before display: Forgejo quotes an unknown token back as `access token does not exist [sha: <TOKEN>]`, so a revoked or expired token would have appeared verbatim in the terminal dock, and from there in screenshots and copied output.
- Documented the `413` case for instances published through a proxy or tunnel with a request-body cap (100 MB on Cloudflare Tunnel's free tier): the initial history push is what hits it, and pushing over SSH — including through a local port forward — is the way around it.
- Added a Forgejo block to the `Remote` pane: check server, save token, who am I, list repositories, use repository, create repository, token page, forget token, plus an `ssh -T Forgejo` probe that honours a non-standard SSH port.
- Replaced the `Remote` pane dropdowns with button rows in the top row: platform (GitHub / GitLab / Codeberg / Forgejo / Gitea / Custom host), URL type (SSH key / HTTPS token) and new-repository visibility. Self-hosted platforms reveal server address and SSH port fields; cloud platforms hide them.
- Generalised remote URL building: `git@host:owner/repo.git` on port 22, `ssh://git@host:PORT/owner/repo.git` otherwise, and `https://host/owner/repo.git` for the HTTPS variant, for every platform rather than three hard-coded hosts.
- Extended Auth Doctor with a Forgejo section: configured credential helpers, per-host reachability, whether a token is stored and whether it is still valid, SSH probes for each configured host/port, and provider labels for self-hosted remotes instead of a bare hostname.
- Changed FZF and portable PowerShell bootstrap resolution to try the stable `releases/latest` URL and its deterministic asset first, using GitHub REST API metadata only if direct resolution or download fails.
- Turned Git commands into dark borderless chips and added interaction-only outlines: azure for safe/read operations and amber for caution/danger operations; command labels use the project's native secondary text color (`#E8E6DC` in Code Dark) and a matching dark neutral on light themes.
- Extended the same individual chip treatment to MIRROR/tree-filter checkboxes, Project/GIT COPY/Docs selectors with their folder buttons, and the six Open actions without darkening their parent sections; checkbox labels use the same command text color.
- Reworked the right workbench into a two-row tab grid: `Quick`, `Branch`, `Editor`, `Diff`, `Storage` over `Remote`, `Basket`, `Reader`, `History`, `Details`; Safety Scan now lives inside `Storage`.
- Added Material icons to right tab headers and delayed tooltips for command buttons and tabs.

- Renamed the Structure tree projection toggle from `MIRROR` to `GIT COPY`.
- Added a dedicated `Branch` pane for branch status, branch/tag commands, merge/revert/cherry-pick, stash and branch-danger templates, removing those duplicate/risky commands from `Quick`.
- Added Remote form fields for platform, remote name, login/group, repository and full URL, with a 20-item recent-value cache per field and explicit use buttons so cached values never overlay typed input.
- Added safer multi-remote Git operations: `push origin` uses `--follow-tags`, `push all remotes` iterates Git remotes in Python, `apply remotes.json` applies enabled remotes, and `origin push URLs` can configure one-origin multi-platform publishing.
- Updated the Structure panel with `PROJECT`, `GIT COPY`, `DOCS` tree-layer switches, folder pickers, a compact selected-path header badge and a nearby clear control.
- Added portable relative path handling for `config/projects.json`: relative paths resolve from the Hub Manager root, picker/scanner writes relative paths when possible, and the bundled Hub Manager project uses `"source_path": "."`.
- Refined the left control panel into `Open`, `MIRROR`, `SOURCE ACTIONS`, `PROJECT ACTIONS` and `SUPPORT` sections, with active-project open buttons, registry-wide batch operations and selected-tree actions separated.
- Added the uppercase `SOURCE` badge for registry-wide Batch Git Status: projects / clean / dirty / errors.
- Kept `Quick -> GIT LOCAL` focused on high-frequency local diagnostics: `init`, `status`, Git root lookup, short log and reflog. `user config` lives in the Auth block inside `Remote`, full config-origin diagnostics live in Git backup/maintenance, and graph history stays in `History`.
- Added `Clone Source` to `SOURCE ACTIONS` as a Source-container clone command template.
- Documented the near-complete everyday Git coverage: local diagnostics, inspect/diff, selected-path staging, commits/tags, remotes, branch/switch, stash, history, integration, backup/maintenance, CI/CD observation and clone.
- Added Support maintenance action `Clean projects.json` to remove stale or duplicate project registry records without deleting any project folders.
- Removed the stale sample Disk Auditor project seed from shipped project configs and launcher examples.
- Added maintained RU/EN documentation editions for README, user guide, technical specification and agent contract.
- Generated dark/light-sand PDF mirrors for the maintained RU/EN documentation set and allowed those themed PDF docs through `.gitignore`.

- Applied the first Opus audit hardening batch: algorithm-tagged hash digests, unified compare-mode defaults, stricter empty-mirror validation, safer path-opening errors, relative terminal-render import and Actions pane command ordering by usage frequency/destructiveness.
- Added explicit BLAKE3 backend verification to the portable builder and the GUI support command panel, with project-scoped log output outside Source/Hub/Docs trees.
- Added read-only BLAKE3 Source/Hub mirror verification with `same`, `changed`, `missing` and `extra` reporting in dated project logs.
- Clarified documentation that Docs/Obsidian/LogSeq folders are not BLAKE3 verification targets because Hub Data is the canonical technical mirror.
- Reworked `CODEX_PROMPTS.md` from early development prompts into a current Codex prompt index and removed stale early-stage wording from active docs.

0.2.3-dev — 2026-05-28

- Filled the right workbench into separate tabs: Actions, Basket, Editor, Preview, Diff, History, Metadata, Auth, Safety and Storage.
- Reworked Actions into a Material-icon Git command grid with centered section labels and no emoji command labels.
- Added Basket as its own pane with longer commit fields, manual version input and version-tag/checkpoint controls.
- Added Markdown Editor pane with CodeMirror preference and textarea fallback.
- Added RedLine Diff, History and Metadata panes for selected paths and repo views.
- Added Auth buttons for Check Auth, `gh auth login`, `glab auth login`, SSH probes, Windows Credential Manager, VS Code and GitKraken folder.
- Added Safety and Storage support panes with summaries, raw JSON and workspace generation/check actions.
- Added HTML/ANSI terminal rendering with Windows-friendly output decoding.
- Refreshed documentation around the current UI state and clarified that Markdown remains canonical while generated PDF copies are request-only.
- Renamed the user-facing docs layer from Obsidian to neutral Docs, with VS Code folders, file managers, Markdown apps and sync tools all treated as valid consumers; Obsidian/LogSeq folders are optional app metadata only.
- Added a Hub profile guard for UI staging/committing so Hub Manager commits use the same extension/mask allowlist as the active projection profile.

- Expanded Dev-Git projection profiles with repository metadata, lock files, formatter/linter configs and common source formats, while excluding machine-local overrides, generated build output, minified bundles, source maps and binary/runtime payloads.
- Documented the project/local boundary: shared `.vscode` project files may be mirrored, while local overrides such as `config/apps.local.json` stay out of Hub Projection and Hub Manager-created commits.
- Hardened `cleanup_project.cmd` for pre-release source cleanup: generated caches, binaries, archives, local overrides and secrets are removed while PDF documentation artifacts are preserved.
- Pruned obsolete Markdown drafts from the donor/porting phase so pre-release docs describe the current product rather than early exploration notes.

v0.1.0-skeleton — 2026-05-27

- Created initial Audion Hub Manager skeleton.
- Added NiceGUI commander layout.
- Added JSON project/profile/remotes config examples.
- Added projection MIRROR engine with hybrid BLAKE3 logic.
- Added post-MIRROR `.gitkeep` handling.
- Added Git wrapper primitives.
- Added Codex continuation docs.

0.2.0-planning

- Added GitHub/GitLab authentication strategy.
- Added external-credentials-only policy.
- Added Auth Doctor CLI skeleton.
- Added storage layout decision: Manager / Hub Data / Docs separation.
- Added Codex prompts for Auth Doctor and storage layout UI.

0.2.1-planning

- Added Disk Auditor MIRROR safety requirements.
- Added profile-level `dry_run_default`, `mirror_scope`, `require_include_filter`, `min_include_globs`, and `delete_after_successful_copy` semantics.

- Added explicit CLI `--apply` for real MIRROR writes when default profile is dry-run.
- Hardened projection apply into copy/touch-before-delete phases.
- Added `strict_blake3` tests for same-size/same-mtime/different-content detection.
- Added safety tests for mask-required profiles and delete-phase skipping.

0.2.1-planning — mirror safety contracts

- Added upstream mirror safety review notes.
- Added projection safety regression tests.
- Standardized phased MIRROR apply: copy/touch first, delete only after clean phases.
- Added/kept `strict_blake3` semantics for same-size/same-mtime content drift.
- Kept `safe_blake3` as Disk Auditor hybrid/fast mode.
- Switched default projection profiles to `strict_blake3` for Hub projections.
- Preserved `.gitkeep` generation after MIRROR.
- Kept GitHub/GitLab credential policy external to Hub Manager.

0.2.2-planning

- Added lightweight Safety Scan core/CLI/UI guard for secret-like files, token patterns, private key blocks, heavy extensions, and large files.
- Added regression tests for safety classification, skipped generated directories, missing roots, and redacted secret-pattern findings.
- Revised project docs and AGENTS RU/EN pairs around current UI direction: Material icon buttons, separate Quick / Basket / Editor panes, and explicit editor-save policy.